

10 Recursive Backtracking

In the previous two chapters we saw efficient divide and conquer algorithms for several problems, including sorting an array, searching a sorted array, and multiplying large numbers. In this chapter, we will explore divide and conquer algorithms based on *recursive backtracking*, a general technique that we can apply whenever the solution to a problem can be generated by making a bunch of small choices. Unlike the divide and conquer algorithms we've seen previously, the ones in this chapter will not be very efficient. However, they will prepare us to learn about efficient dynamic programming algorithms in Chapter 11.

10.1 Subset Sum

In the subset sum problem, we are given a set $S \subseteq \mathbb{Z}^+$ of positive integers and a non-negative integer $k \in \mathbb{Z}_{\geq 0}$, and want to determine whether or not there exists a set $T \subseteq S$ with $\sum_{x \in T} x = k$ (i.e. whether there is a subset of S whose elements sum to exactly k). For example, if $S = \{2, 3, 7, 11, 19\}$ and $k = 24$, we should return True, because $\{2, 3, 19\} \subseteq S$ and $2 + 3 + 19 = 24$.

How can we solve this problem with a divide and conquer approach? (i.e. how can we convert the problem to a smaller version of itself?). One natural idea is to choose a first element x to include in our subset, and then recursively choose the rest of the elements so that they sum to $k - x$. We give a recursive algorithm based on this approach below:

```

SUBSETSUMSOLVER( $S, k$ )
  Remove from  $S$  all elements which are greater than  $k$ 
  If  $S = \emptyset$ :
    If  $k = 0$ :
      Return True
    Return False
  For each  $x \in S$ :
    If SUBSETSUMSOLVER( $S \setminus \{x\}, k - x$ ):
      Return True
  Return False

```

Theorem 52. *For every finite $S \subset \mathbb{Z}^+$ and $k \in \mathbb{Z}_{\geq 0}$ SUBSETSUMSOLVER(S, k) returns whether or not there exists $T \subseteq S$ with $\sum_{x \in T} x = k$. Furthermore SUBSETSUMSOLVER runs in time $\Omega(n!)$, where $n = |S|$.*

Proof. We prove the correctness of SUBSETSUMSOLVER by induction on k .

- Base case: If $k = 0$, then the first line will remove all elements from S , so S will be the empty set and the algorithm will immediately return True, which is correct, since every set has a subset summing to 0, namely the empty set.
- Induction hypothesis: For every $0 \leq k' < k$ and every finite $S' \subset \mathbb{Z}^+$, SUBSETSUMSOLVER(S', k') returns whether or not there exists $T \subseteq S'$ with $\sum_{x \in T} x = k'$.
- Suppose $k > 0$. We first note that removing the elements of S which are greater than k cannot change whether or not S has a subset summing to k . Since all elements of S are positive, any subset which includes an element $x > k$ must have a sum of at least x , and thus cannot have a sum of k .

If S is empty after these removals, then it cannot have a subset summing to k , since $k \neq 0$, so the algorithm correctly returns False. Otherwise, we consider each element of S for inclusion in our subset. We note that if $T \subseteq S$ sums to k and y is an element of T , then $T \setminus \{y\}$ is a subset of $S \setminus \{y\}$ which sums to $k - y$. Since $y > 0$, $k - y < k$. Thus, by our IH, when our loop reaches y , SUBSETSUMSOLVER($S \setminus \{y\}, k - y$) will return True, and so will SUBSETSUMSOLVER(S, k).

Conversely, if SUBSETSUMSOLVER($S \setminus \{y\}, k - y$) returns true for some y , then by our IH we know that there exists some $T' \subseteq S \setminus \{y\}$ with $\sum_{x \in T'} x = k - y$. Then $T' \cup \{y\}$ is a subset of S summing to k , so when the algorithm returns True, it is correct.

We have shown that if it has not returned beforehand the algorithm returns True in the loop if and only if S has a subset summing to k . Therefore, if we reach the end of the loop and none of the recursive calls have returned True, then S does not have a subset summing to k and the algorithm correctly returns False.

Now we analyze the runtime. We only need to prove a lower bound, so we can choose a particular class of inputs to focus on (one that we think represents a worst-case for the algorithm). Suppose that $k = 1 + \sum_{x \in S} x$. Then there is no subset of S summing to k and we will never discard elements for being greater than k , so every subproblem will make a recursive call for every element in the set.

Each recursive call decreases the size of our set by 1, so in total we make:

- 1 call with a set of size n
- n calls with a set of size $n - 1$
- $n(n - 1)$ calls with a set of size $n - 2$
- ...
- $\prod_{j=0}^{i-1} (n - j)$ calls with a set of size $n - i$

In particular, this means that we make $n!$ calls to our base case where the size of our set is 0. Thus, we can lower bound the total runtime with $n!$. □

A runtime of $\Omega(n!)$ is extremely inefficient. Let's see if we can do a little bit better. SUBSETSUM-SOLVER is slow in part because it considers the same subset many times. For example, the algorithm considers adding 1 first then 2 separately from adding 2 first, then 1. We can avoid this redundancy by applying an arbitrary ordering to our set, and only considering adding elements in this fixed order. One natural way to do this is to represent our set S as an array A . Since we are now considering the elements in a fixed order, the choice that we make is not “which element should we include next” but “should we include $A[i]$ in the subset?” Including $A[i]$ should only be an option if $A[i] \leq k$, otherwise we're forced to skip it.

Since we are trying to be efficient here, we want to avoid overhead from modifying and copying an array, so instead, we will keep the same array A (which we implicitly pass by reference) and pass our recursive calls an extra parameter i representing the index of the array to start at. Based on our discussion in the previous paragraph, we can write a recursive function $SS(A, k, i)$ which determines whether or not $A[i : n]$ has a subset summing to k :

$$SS(A, k, i) = \begin{cases} \text{True} & \text{if } k = 0 \\ \text{False} & \text{if } i > n \text{ and } k > 0 \\ SS(A, k, i + 1) & \text{if } i \leq n, k > 0 \text{ and } A[i] > k \\ SS(A, k - A[i], i + 1) \vee SS(A, k, i + 1) & \text{otherwise} \end{cases}$$

This recursive function immediately translates into a recursive algorithm

```

SECONDSUBSETSUMSOLVER( $A[1, \dots, n], k, i = 1$ )
  If  $k = 0$ :
    Return True
  If  $i > n$ :
    Return False
  If  $A[i] > k$ :
    Return SECONDSUBSETSUMSOLVER( $A, k, i + 1$ )
  Return SECONDSUBSETSUMSOLVER( $A, k - A[i], i + 1$ )  $\vee$  SECONDSUBSETSUMSOLVER( $A, k, i + 1$ )

```

Theorem 53. For every $n, k \in \mathbb{Z}_{\geq 0}$ and every array A of n positive integers, $\text{SECONDSUBSETSUMSOLVER}(A, k)$ returns whether or not there exists a subset T of the elements of A with $\sum_{x \in T} x = k$. Furthermore $\text{SECONDSUBSETSUMSOLVER}$ runs in time $\Theta(2^n)$.

Proof. We will sketch the correctness proof rather than using a full induction proof, since the argument is similar to what we already showed in Theorem 52. We note that if the algorithm returns True, then it found a subset of the elements of A which sum to k (we can trace up the call stack from the call to `SECONDSUBSETSUMSOLVER` which returned True and see where the algorithm recursed on `SECONDSUBSETSUMSOLVER(A, k - A[i], i + 1)`).

On the other hand, if S has a subset T summing to k , then our algorithm will find it through the chain of function calls where it recurses on `SECONDSUBSETSUMSOLVER(A, k - A[i], i + 1)` (corresponding to adding $A[i]$ to our subset) whenever $A[i] \in T$ and `SECONDSUBSETSUMSOLVER(A, k, i + 1)` whenever $A[i] \notin T$ (corresponding to omitting $A[i]$ from our subset). The algorithm will not know what T is, but we are trying all options through recursive brute force, so one chain of recursive calls will happen to choose exactly the elements in T .

Now we justify the runtime. `SECONDSUBSETSUMSOLVER` does $\Theta(1)$ work outside of its recursive calls, and makes 2 recursive calls to inputs of size $n - 1$. In the worst case (e.g. if $k > \sum_{i=1}^n A[i]$, the algorithm will not return until the base case where $i = n + 1$, so the runtime satisfies the recurrence $T(n) = 2T(n - 1) + \Theta(1)$.

This is not a recurrence that we can directly apply the master theorem to, since we are not dividing the input size by a constant. However, we can still use the theorem if we are a bit clever. Define a new function T' whose domain is powers of 2 by $T'(x) := T(\log_2 x)$. Then, by our recurrence for T , T' satisfies the recurrence

$$T'(x) = 2T((\log_2 x) - 1) + \Theta(1) = 2T'(x/2) + \Theta(1),$$

since $\log_2(x/2) = \log_2(x) - 1$. The master theorem tells us that this recurrence solves to $T'(x) \in \Theta(x)$, and thus $T(\log_2 x) \in \Theta(x)$. Plugging in n for $\log_2 x$ we get that $T(n) \in \Theta(2^n)$. $\square$

`SECONDSUBSETSUMSOLVER` is much faster than `SUBSETSUMSOLVER`, but the runtime is still exponential in n . Next chapter when we talk about dynamic programming, we will see a way to further improve the runtime of this algorithm.

10.2 Longest Increasing Subsequence

Let's see another example of a problem that we can solve through recursive backtracking. First we need to define the notion of a subsequence. For a sequence/string/array $x = x_1, x_2, \dots, x_n$, a *subsequence* is a sequence $x_{i_1}, x_{i_2}, \dots, x_{i_k}$, where $1 \leq i_1 < i_2 < i_3 < \dots < i_k \leq n$. That is, a subsequence of x is a list of elements from x in the same order that they appear in x . Unlike a *substring*, however, the elements do not have to be consecutive in x , the subsequence can skip over elements. For example, 1,3,7 is a subsequence of 1,2,3,5,6,7,9.

Now we can define the problem we are interested in. In the longest increasing subsequence problem we are given as input an array A and we want to find the length of the longest subsequence of A where all the elements are in increasing order. For example if $A = [10, 5, 3, 4, 1, 6, 12, 9, 20, 2, 3, 4]$ then a longest increasing subsequence is 3, 4, 6, 12, 20, which has length 5.

As we did in `SECONDSUBSETSUMSOLVER`, we can view a solution to longest increasing subsequence as constructed from a sequence of binary choices of “include $A[i]$ or skip it”. This suggests a recursive structure where we relate the length of the longest increasing subsequence of $A[i, \dots, n]$ to the length of the longest increasing subsequence of $A[i + 1, \dots, n]$.

We have to be a bit more clever here, however, because it's not immediately clear how knowing the length of the longest increasing subsequence of $A[i + 1, \dots, n]$ helps us find the length of the longest increasing subsequence of $A[i, \dots, n]$.

Using the example array $A = [10, 5, 3, 4, 1, 6, 12, 9, 20, 2, 3, 4]$ from above, suppose that we are at index $i = 3$ with $A[i] = 3$ and a recursive call tells us that the longest increasing subsequence of $A[4 : n]$ has length 4. This tells us that the length of longest increasing subsequence of $A[3 : n]$ is either 4 (if the best subsequence for $A[3 : n]$ is the same as the best one for $A[4 : n]$) or 5 (if we can add 3 to the beginning of a longest increasing subsequence for $A[4 : n]$), but it's not enough information to tell whether 4 or 5 is the right answer.

Even if our recursive call returned an actual maximal subsequence, we still wouldn't have enough information to solve the problem. For example, 1, 2, 3, 4 is a longest increasing subsequence of $A[4 : n]$, and we clearly cannot add 3 to the beginning of this sequence. Nevertheless, the longest increasing subsequence of $A[3 : n]$ does have length 5, because we can add 3 to the beginning of an alternative longest increasing subsequence for $A[4 : n]$, such as 4, 6, 12, 20.

What we need is some way to tell our recursive call when we have already chosen $A[i]$ for our subsequence, and all subsequent elements must be larger than this. In the subset sum problem, we were able to accomplish the goal of letting the recursive call know about our previous choices by modifying the k parameter. In this case, our problem doesn't have a second integer parameter, but that doesn't mean we can't add one! By introducing a second parameter representing "the previous element that we picked", we can give a recurrence for longest increasing subsequence.

Let $LIS(i, j)$ denote the length of the longest increasing subsequence of $A[i, \dots, n]$ where every element is larger than $A[j]$. We give the following recurrence:

$$LIS(i, j) = \begin{cases} 0 & \text{if } i > n \\ LIS(i + 1, j) & \text{if } i \leq n \text{ and } A[i] \leq A[j] \\ \max\{LIS(i + 1, j), 1 + LIS(i + 1, i)\} & \text{otherwise} \end{cases}$$

Here our recurrence represents the choice of whether or not to include $A[i]$ in our subsequence. If $A[i] \leq A[j]$, we have no choice, we have to skip $A[i]$. If $A[i] > A[j]$, we can include it, in which case we increase the length of our sequence by 1, but change our lower bound on future elements to $A[i]$, or we can skip it. We choose whichever option gives a longer subsequence. We give the following pseudocode for LIS:

```

LIS-SOLVER( $A[1, \dots, n], i, j$ ):
  If  $i > n$ :
    Return 0
  If  $A[i] \leq A[j]$ :
    Return LIS-SOLVER( $A, i + 1, j$ )
  Return max(LIS-SOLVER( $A, i + 1, j$ ), 1 + LIS-SOLVER( $A, i + 1, i$ ))

```

Note that the pseudocode directly translates the recurrence. This time there is not really a natural default value for the extra parameter j that we have added, so we need to specify how to use our recursive algorithm for solving our modified version of the function (where we require all elements of our sequence to be larger than $A[j]$) in order to solve the original problem.

In this case, to solve the original problem, we need to compute $\max_{1 \leq i \leq n} (1 + \text{LIS-SOLVER}(A, i + 1, i))$. That is, we take the max, over all choices of first element, of the longest increasing subsequence we can get starting with that element.

Once again, this algorithm runs in exponential time. Just like `SECONDSUBSETSUMSOLVER`, LIS satisfies the recurrence $T(n) = 2T(n - 1) + \Theta(1)$ in the worst case, so the runtime is $\Theta(2^n)$ by a similar analysis.

Note that if we wanted to we could easily modify our algorithm to return the actual longest subsequence (instead of just the length). We would simply change our base case to return an empty sequence and instead of returning $\max(\text{LIS-SOLVER}(A, i + 1, j), 1 + \text{LIS-SOLVER}(A, i + 1, i))$ we would return the longer of the sequence returned by $\text{LIS-SOLVER}(A, i + 1, j)$ and the sequence returned by $\text{LIS-SOLVER}(A, i + 1, i)$ with $A[i]$ inserted at the front.

10.3 Designing Recursive Backtracking Algorithms

The general approach to designing recursive backtracking algorithms is to solve a problem through *recursive brute force*. We need to:

1. Determine how to construct a solution to the problem by making a sequence of small choices (i.e. figure out the recursive structure of solutions)

2. Determine what information our algorithm needs to have about the previous choices in order to make the next choice in the sequence validly. Often this will require modifying the original problem to add an extra parameter.
 - For subset sum the information about our previous choices was communicated entirely by changing k
 - For longest increasing subsequence we had to pass along the extra parameter j keeping track of the last element we added to the sequence
 - Keeping this summary as small as possible will help us convert our recursive backtracking algorithm to an efficient DP algorithm later. This is why we chose to have our longest increasing subsequence algorithm pass along only the previous element, not all of the elements chosen so far.
3. Clearly state the version of the problem that you are solving. For example, for longest increasing subsequence we defined our problems by saying “Let $LIS(i, j)$ denote the length of the longest increasing subsequence of $A[i, \dots, n]$ where every element is larger than $A[j]$.”
4. Construct a recursive function describing the solution to your chosen modification of the problem in terms of the solutions to smaller subproblems.
 - Typically this recursive function will have the structure:
 - Try all possible options for the first choice in our sequence of choices
 - For each possible option, recursively make the rest of the choices in a way that optimizes our objective
 - Combine the result of the recursive calls in order to determine the best first choice.
5. (Optional) Write recursive pseudocode based on your recurrence. Often the pseudocode is almost word-for-word the same as the recurrence, so this step is not strictly necessary. Sometimes there may be important implementation details to discuss, however, and if you aren’t pressed for time then it never hurts to explicitly give pseudocode.
6. Describe how to use your recursive function to solve the original problem
 - That is, describe what value(s) to use for any additional parameters you have added.
 - e.g. for longest increasing subsequence we took $\max_{1 \leq i \leq n} (1 + LIS\text{-}SOLVER(A, i + 1, i))$.

10.4 Longest Common Subsequence

One simple problem about subsequences with a variety of practical applications is *longest common subsequence*. In this problem we are given two arrays $A[1, \dots, m]$ and $B[1, \dots, n]$ and we want to find the longest sequence which is a subsequence of both A and B . For example, if $A = [1, 7, 3, 12, 15, 2, 2, 1, 7]$ and $B = [3, 1, 12, 14, 9, 2, 5, 1, 7, 16]$ then a longest common subsequence is $[3, 12, 2, 1, 7]$, which has length 5 ($[1, 12, 2, 1, 7]$ is also a longest common subsequence).

Solving longest common subsequence is useful for comparing two versions of a file and determining what has changed between them (e.g. in a version control system like git). The longest common subsequence problem also arises in biology, when trying to compare two DNA sequences for common ancestry.

Let’s apply the recursive backtracking techniques that we have discussed to solving longest common subsequence. For simplicity, we will focus on finding the *length* of the longest common subsequence. We need to figure out how to view constructing a longest common subsequence as making a series of small choices.

A natural approach is to say that each choice is one element of the common subsequence. So our recursive function should consider all possible choices for the first element of the subsequence and then recursively compute which choice maximizes the length of the rest of the subsequence.

Every pair of a character from A and a matching character from B (not necessarily at the same index) forms a potential starting point for our common subsequence. In order to make sure that the remaining elements that we choose all occur after the first elements we pick, our recursive call needs to pass along the indices of the elements that we picked in both A and B .

Thus, the version of the problem that we are solving involves two extra index parameters, which we can call i and j . Let $LCS(i, j)$ denote the length of the longest common subsequence of $A[i : m]$ and $B[j : n]$. To simplify the description of our recurrence, we define an auxiliary function I . Let $I(i, j) \subseteq \{i, i + 1, \dots, m\} \times \{j, j + 1, \dots, n\}$ be defined by

$$I(i, j) := \{(i', j') \mid i \leq i' \leq m, j \leq j' \leq n \text{ and } A[i'] = B[j']\}.$$

That is, $I(i, j)$ is the set of all pairs of indices (i', j') in our range of valid indices where $A[i'] = B[j']$. Then LCS satisfies the following recurrence:

$$LCS(i, j) = \begin{cases} 0 & \text{if } I(i, j) = \emptyset \\ \max_{(i', j') \in I(i, j)} 1 + LCS(i' + 1, j' + 1) & \text{otherwise} \end{cases}$$

This recurrence yields the following recursive algorithm:

```

LCS-SOLVER( $A[1, \dots, m], B[1, \dots, n], i = 1, j = 1$ ):
   $I \leftarrow \emptyset$ 
  For  $i'$  from  $i$  to  $m$ :
    For  $j'$  from  $j$  to  $n$ :
      If  $A[i'] = B[j']$ 
        Add  $(i', j')$  to  $I$ 
  If  $I = \emptyset$ :
    Return 0
  Return  $1 + \max_{(i', j') \in I} \text{LCS-SOLVER}(A, B, i' + 1, j' + 1)$ 

```

The solution to our original problem is simply $\text{LCS-SOLVER}(A, B)$ (which sets i and j to 1).

In the worst case where all the elements of A and B are identical, this algorithm again takes exponential time (the analysis is a little more involved than the previous two problems, and not particularly important). Next chapter we will see how to convert all of the exponential time recursive algorithms we have seen in this chapter into polynomial time dynamic programming algorithms.

10.5 Comprehension Questions

1. What does $\text{SUBSETSUMSOLVER}(\{1, 2, 3, 4, 5\}, 13)$ return?
2. True or false? $\text{SECONDSUBSETSUMSOLVER}$ runs in time $O(n^2)$.
3. What is a longest increasing subsequence of $A = [3, 4, 1, 7, 12, 15, 8]$?
4. True or false? If $LIS(i, j)$ is the length of the longest increasing subsequence of $A[i, \dots, n]$ where every element is larger than $A[j]$ then LIS satisfies the recurrence

$$LIS(i, j) = \begin{cases} 0 & \text{if } i > n \\ \max\{LIS(i + 1, j), 1 + LIS(i + 1, i)\} & \text{otherwise} \end{cases}$$

5. What is a longest common subsequence of $A = [2, 3, 8, 7, 1]$ and $B = [1, 7, 8, 7, 3, 2, 1]$?

10.6 Exercises

1. Give a recursive backtracking algorithm that takes as input an array $A[1, \dots, n]$ of integers and computes the length of the longest *decreasing* subsequence of A . You should justify the correctness of your algorithm, but you do not need to analyze the runtime.

2. We define a *valley* as a sequence of integers $v_1, \dots, v_k$ with the following three properties:

- k is odd
- For every $1 \leq i \leq (k-1)/2$, we have that $v_i \geq v_{i+1}$
- For every $(k+1)/2 \leq i \leq k-1$, we have that $v_i \leq v_{i+1}$.

For example, the sequence 5, 4, 4, 7, 10 would be a valley, but the sequence 7, 6, 5, 8, 9, 11, 13 would not be. Give a recursive backtracking algorithm which takes as input an array $A[1, \dots, n]$ of positive integers and returns the longest subsequence of A which is a valley. You should justify the correctness of your algorithm, but you do not need to analyze the runtime.

3. You are planning a road trip between cities A and B . Fortunately, they are connected by a long straight highway. Unfortunately, you are going to have to stop for fuel several times, and all of the gas stations on this highway use a very unusual pricing system. Gas is sold by the tank, meaning that you pay the same fixed amount to refill your tank, whether that takes one gallon or twenty.

Suppose that you are given d , the distance in miles that your car can travel on a full tank of gas, as well as two arrays $G[1, \dots, n]$ and $P[1, \dots, n]$, where $G[i]$ is the location of the i th gas station, given as a distance in miles from your starting point, and $P[i]$ is the price (in dollars) to fill your tank at the i th gas station. Assume that your trip begins at the first gas station (so $G[1] = 0$) with an empty tank of gas, and ends at the n th gas station (where you do not need to refuel). Assume that it is possible to complete the trip (i.e. there is no stretch of d miles with no gas stations). Give a recursive backtracking algorithm to calculate the minimum amount that you can pay for gas while making this trip. You should justify the correctness of your algorithm, but you do not need to analyze the runtime.

4. Susan is going door to door in her neighborhood selling boxes of cookies. Susan's cookies come in two different kinds. She has x boxes of chocolate chip cookies and y boxes of oatmeal raisin cookies. Susan's neighborhood consists of n houses arranged in a straight line. Each household is willing to buy at most one box of cookies, and they each value the two kinds of cookies differently. Fortunately, Susan has done extensive market research, and has compiled two arrays, $A[1, \dots, n]$ and $B[1, \dots, n]$, where $A[i]$ is the price (in cents) that the i th house will pay for a box of chocolate chip cookies, $B[i]$ is the price that they will pay for a box of oatmeal raisin cookies (You may assume that all of these prices are non-negative integers).

Susan needs to decide which of her two kinds of cookies to offer at each house in order to maximize the revenue she earns from her limited cookie stash. Give a recursive backtracking algorithm to help her with this problem. Your algorithm should take as input the values x and y and the arrays A and B . Your algorithm should output a length n array where each entry is either "chocolate chip", "oatmeal raisin", or "nothing", and for each i the i th entry describes which type of cookie Susan should sell at the i th house in order to maximize the total amount she earns from selling these cookies. You should justify the correctness of your algorithm, but you do not need to analyze the runtime.

5. Suppose you are given a list of n positive integers separated by plus and minus signs, for example

$$5 + 2 - 3 + 7 - 4 - 12$$

You can change the value of this expression by adding parentheses in different positions. For example:

$$\begin{aligned} 5 + 2 - 3 + 7 - 4 - 12 &= -5 \\ 5 + 2 - (3 + 7) - (4 - 12) &= 5 \\ 5 + (2 - 3 + 7 - (4 - 12)) &= 19 \end{aligned}$$

Give a recursive backtracking algorithm to find a placement of parentheses which maximizes the value of the expression. You may only use the parentheses to group additions and subtractions, you are not allowed to create implicit multiplication (e.g. parenthesizing $5 - 2$ as $5(-2)$ is not allowed). You should justify the correctness of your algorithm, but you do not need to analyze the runtime.